CENTRO DE ESTUDIOS SUPERIOR DEL NOROESTE

Actividad: Python, Entornos Virtuales y Patrones de Diseño

Creado Por:	Bladimir Mejía Hernández
Materia	Patrones de Diseño
Docente	Miguel Angel Cardona
Carrera	Ingeniería en Desarrollo de Software
Grupo	08IDESMA
Fecha	9/03/2026

Introducción

En esta actividad analicé tres temas fundamentales dentro del desarrollo de software con Python: la gestión de entornos virtuales, la implementación de un generador de códigos QR y la investigación de patrones de diseño. El objetivo fue comprender no solo cómo resolver el reto técnico planteado en clase, sino también justificar por qué estas prácticas son importantes en un contexto profesional y académico.

1. Gestión de Entornos Virtuales

¿Qué es un entorno virtual?

Un entorno virtual es un espacio aislado dentro del proyecto que permite instalar librerías y dependencias sin afectar la instalación global de Python del sistema. Su uso evita conflictos entre versiones de paquetes y facilita que el proyecto sea portable, reproducible y más fácil de compartir con otros compañeros o desplegar en otro equipo.

Proceso de creación y activación

Para crear el entorno virtual, primero abrí la terminal dentro de la carpeta del proyecto y ejecuté el siguiente comando:

```
python -m venv venv
venv\Scripts\activate
pip install qrcode[pil]
```

Cuando el entorno virtual queda activado, normalmente aparece el prefijo (venv) al inicio de la terminal. Esto indica que todas las instalaciones de librerías se harán dentro del proyecto y no a nivel global.

¿Para qué sirve el comando python -m venv venv?

El comando ejecuta el módulo venv de Python para crear una instalación aislada dentro de una carpeta llamada venv. La primera palabra python invoca el intérprete; la opción -m le dice a Python que ejecute un módulo; el módulo venv crea el entorno; y el último venv es el nombre de la carpeta donde se guardará.

¿Por qué es mala práctica instalar librerías globalmente?

- Porque diferentes proyectos pueden requerir versiones distintas de una misma librería.
- Porque aumenta el riesgo de conflictos entre dependencias.
- Porque ensucia la instalación principal de Python del sistema operativo.
- Porque dificulta compartir el proyecto y reproducirlo en otra computadora.

2. Implementación de Generador de QR

Descripción general de la solución

Para resolver el reto planteado en clase utilicé la librería qrcode de Python. La solución consistió en importar la librería, definir la información que se desea convertir en QR, configurar el objeto QR y generar una imagen en formato PNG. Este enfoque es práctico

porque en pocas líneas de código permite producir un resultado funcional y fácil de verificar con la cámara de un celular.

Código utilizado

```
import qrcode
import os

# Crear carpeta si no existe
os.makedirs('qr_codes', exist_ok=True)

# Lista de datos para generar QRs
datos = [
    'https://github.com',
    'https://google.com',
    'https://python.org',
    'https://wikipedia.org',
    'https://stackoverflow.com',
]

# Generar QRs
for i, url in enumerate(datos, 1):
    qr = qrcode.QRCode(
        version=1,
        error_correction=qrcode.constants.ERROR_CORRECT_H,
        box_size=10,
        border=4,
    )
    qr.add_data(url)
    qr.make(fit=True)

    img = qr.make_image(fill_color="black", back_color="white")
    img.save(f'qr_codes/qr_{i}.png')
    print(f"✓ QR {i} creado: {url}")

print("\n✓ Todos los QRs fueron creados en la carpeta 'qr_codes/'")
```

Flujo del código

Etapa	Descripción	Propósito
Importación	Se importa la librería qrcode.	Permite acceder a las funciones de generación del QR.
Configuración de data	Se define una variable con el enlace o texto a codificar.	Establece el contenido que será leído por el celular.
Creación del objeto	Se construye QRCode con version, box_size y border.	Controla tamaño, resolución y borde del código.
Generación de imagen	Se agrega la data, se ajusta el tamaño y se crea la imagen.	Produce el código QR con formato visual legible.
Guardado	Se usa save para crear el archivo PNG.	Conserva el resultado en un archivo reutilizable.

Problema técnico específico resuelto

El reto técnico consistía en generar un código QR que realmente pudiera escanearse y entregar el contenido correcto. Para lograrlo, fue clave instalar la librería dentro del entorno virtual, usar un borde suficiente para que el lector reconociera el patrón y guardar el archivo en un formato compatible como PNG. En la práctica, la solución se valida probando el QR con la cámara del celular o con una aplicación lectora.

Ejemplo de código QR generado

Este ejemplo dirige a un enlace web de prueba.

3. Investigación de Patrones de Diseño

Los patrones de diseño son soluciones reutilizables a problemas comunes de arquitectura y organización del código. No son plantillas rígidas, sino enfoques probados que ayudan a construir software más mantenible, escalable y fácil de entender.

Patrón	Definición	Ejemplo de aplicación real
Singleton	Garantiza que una clase tenga una sola instancia y un punto global de acceso.	Una única configuración general o una sola conexión controlada a recursos compartidos.
Factory Method	Define una interfaz de creación de objetos sin especificar la clase concreta que se va a instanciar.	Crear distintos tipos de notificaciones: correo, SMS o push según el contexto.
Observer	Establece una relación uno a muchos en la que varios objetos son notificados cuando otro cambia de estado.	Sistema de notificaciones, suscripciones o actualización automática de interfaces.
Strategy	Permite intercambiar algoritmos o comportamientos en tiempo de ejecución sin modificar la estructura principal.	Seleccionar distintos métodos de pago o distintas reglas de cálculo.
Decorator	Agrega funcionalidades adicionales a un objeto de forma dinámica sin alterar su clase base.	Añadir extras a una bebida, permisos a una vista o filtros adicionales a una respuesta.

4. Evidencias requeridas para la entrega

El documento final que se suba al repositorio debe contener las siguientes evidencias. En este archivo se deja una guía clara de qué debe integrarse para completar la entrega de manera profesional.

- Captura de la terminal con el entorno virtual activado.
- Captura o inserción del archivo Python utilizado para generar el QR.
- Imagen del código QR generado funcionando.
- Comprobación de que el QR dirige a un enlace o muestra un texto válido.
- Archivo: .gitignore para evitar subir la carpeta venv/ al repositorio.

Espacio para evidencia 1

captura real de la terminal con el entorno virtual activado.

```
(venv) C:\Users\test\LABS\lab1\codigofuente>python crear_qrs.py
✓ QR 1 creado: https://github.com
✓ QR 2 creado: https://google.com
✓ QR 3 creado: https://python.org
✓ QR 4 creado: https://wikipedia.org
✓ QR 5 creado: https://stackoverflow.com

✓ Todos los QRs fueron creados en la carpeta 'qr_codes/'
```

Espacio para evidencia 2

Inserta aquí tu captura del script qr.py o de la ejecución del programa.

crear_qrs.py 1, U

crear_qrs.py > ...

```python
import qrcode
import os
# Crear carpeta si no existe
os.makedirs('qr_codes', exist_ok=True)

# Lista de datos para generar QRs
datos = [
    'https://github.com',
    'https://google.com',
    'https://python.org',
    'https://wikipedia.org',
    'https://stackoverflow.com',
]

# Generar QRs
for i, url in enumerate(datos, 1):
    qr = qrcode.QRCode(
        version=1,
        error_correction=qrcode.constants.ERROR_CORRECT_H,
        box_size=10,
        border=4,
    )
    qr.add_data(url)
    qr.make(fit=True)

    img = qr.make_image(fill_color="black", back_color="white")
    img.save(f'qr_codes/qr_{i}.png')
    print(f"✓ QR {i} creado: {url}")

print("\n✓ Todos los QRs fueron creados en la carpeta 'qr_codes/'")
```

5. Reflexión personal

Desde mi punto de vista, la sintaxis de Python me pareció clara, limpia y fácil de entender. Comparado con otros lenguajes, Python permite escribir menos código para lograr resultados útiles en poco tiempo. Esto se nota especialmente cuando se trabaja con librerías ya preparadas, como qrcode, que permiten resolver problemas reales con una curva de aprendizaje accesible. Además, el uso de entornos virtuales ayuda a mantener los proyectos organizados y profesionales, lo cual es una práctica indispensable cuando se trabaja en equipo o se desea subir un proyecto a GitHub de forma ordenada.

Conclusión

En esta actividad pude comprender la importancia de usar entornos virtuales en Python, ya que permiten aislar dependencias y mantener proyectos limpios y reproducibles. También reforcé el proceso de creación de un código QR funcional mediante el uso de librerías externas y comprobé que Python destaca por su productividad y facilidad de uso. Finalmente, el repaso de los patrones de diseño me permitió relacionar teoría con aplicaciones reales de desarrollo de software, lo cual será clave para el examen técnico de la siguiente clase.

Anexo: comandos utilizados

```
python -m venv venv

venv\Scripts\activate

pip install qrcode[pil]

python qr.py

git add .

git commit -m "Agrega actividad Python y PDF"

git push origin main
```